
Adversarial Iterative Constraining for Reducing AI-Detectability

Vongai Mitchell Makuwaza (vongai-mitchell.makuwaza@student.uni-tuebingen.de)¹

Finn Springorum (finn.springorum@student.uni-tuebingen.de)¹

Abstract

The widespread employment of Large Language Models (LLMs) renders the task of detecting potentially harmful and unsupervised AI-generated texts more significant than ever. To examine the underlying capabilities of LLMs in generating human-like texts, we perform an iterative adversarial attack on an out-of-distribution text detector, where we employ a beam search procedure to extend a simple initial base prompt with AI-generated adaptive constraints, which aim to gradually reduce the detector scores on the corresponding LLM outputs. Our results reveal that this procedure is highly successful while still yielding coherent generations, outperforming zero-shot instructions for GPT-5 to humanize the initial generations by a large margin. However, it remains questionable what our results imply due to the multi-dimensional nature of the task setting. Our code and results are available at <https://github.com/FinnSpr/adversarial-constraining>.

1. Introduction

Large Language Models (LLMs) are now being widely used for text generation, and a significant portion of text on the Internet is already LLM-generated (Paredes et al., 2026). Meanwhile, the transparency about LLM usage steadily decreases, and it is likely that it will soon be impossible to distinguish authentic texts from potentially incorrect or otherwise harmful AI texts, given the continuously improving capabilities of LLMs. In this study, we examine how easy it is to receive an LLM output that appears less AI-generated by performing an automated attack against a state-of-the-art (SoTA) LLM text detector, where we iteratively add adaptive constraints to a simple base prompt, thereby fol-

lowing a widely used and obvious workflow that humans employ when using LLMs in their daily or professional life. Specifically, we perform a beam search where every node extends the prompt with an LLM-generated constraint from one of six categories, based on the previous text output and corresponding detector score. This work aims to uncover the hidden potential of LLMs in generating human-like text that could be easily unlocked by ordinary users without AI knowledge.

2. Methods

2.1. Base Prompts & Constraints

We collect 10 one-sentence AI-generated base prompts for each of five different task paradigms: creative writing, factual, opinion, step-by-step guides, and problem solving. Furthermore, we define six vast categories for possible constraints: structural, lexical, stylistic, mimicry, content-level, and imperfection (see appendix 6 for their definitions), restricting the otherwise free choice of adaptive constraints to a specific paradigm.

2.2. Detector Model

We employ a recently presented AI text detector by Zeng et al. (Zeng et al., 2025) that formulates the detection task as an out-of-distribution (OOD) detection problem, where human-written texts are treated as distributional outliers, while machine-generated texts are in-distribution samples. We posit that this formulation is highly appropriate for our strategy to successively reduce the AI-likelihood of the generated texts, since our constraints may try to mitigate the typical properties of machine-generated texts in order to escape from their distribution and thus make them appear as human-like outliers. The authors provide the weights for the DeepSVDD model trained on the DeepFake (Li et al., 2024) and RAID (Dugan et al., 2024) datasets, respectively, claiming that DeepFake focuses on cross-domain and model generalization, while RAID prioritizes attack robustness. They report a SoTA performance on 10K test samples against all previous zero-shot and training-based methods.

However, we validated the performance of these two pre-trained detectors on the Human-AI Parallel English Corpus

¹University of Tübingen, Tübingen, Germany. Correspondence to: Vongai Mitchell Makuwaza <vongai-mitchell.makuwaza@student.uni-tuebingen.de>, Finn Springorum <finn.springorum@student.uni-tuebingen.de>.

dataset (Reinhart et al., 2024), whose creators split 1,000 words of existing human text and prompted an LLM with the first chunk and the instruction to continue this text in the same style. We thus contrast the full 8,290 human texts (second chunk) with the AI-generated counterpart as generated by GPT-4o-2024-08-06. The detector returns a score between 0 and 1, which can be interpreted as the likelihood of the text being AI-generated. Interestingly, while the score distribution of the DSVD model trained on DeepFake exhibits a great contrast, the RAID detector incorrectly assigns a low score to many of the GPT-generated texts (see appendix Figure 12). Therefore, we selected the DeepFake model for the subsequent analysis.

2.3. Generation, Constraint & Judge Model

We use Qwen3.5-9B (Qwen Team, 2026) for the generation of texts given the constrained prompt, the generation of constraints given the previous prompt, generation output, and detector score, and also for a coherence judgment based on a scale from 1 to 5 (prompts in appendix). Besides enabling cheaper computation, we posit that doing both constraining and generation with the same model might yield a more successful attack performance, as the model may know what constraints work best for itself.

2.4. Humanizing Baseline

To establish a point of comparison for our constraint search attack, we evaluate a straightforward "humanization" baseline. This approach tests whether simply instructing a language model to rewrite the text to sound more natural is sufficient to evade the detector, without the need for an iterative constraint-generation pipeline.

For this baseline, we isolate the initial, unconstrained generations at depth 0 and process them using a dedicated humanizing prompt (see appendix 7). The prompt explicitly instructs the model to rewrite the original text to sound more human, natural, and emotionally resonant, while avoiding robotic phrasing and preserving both the core meaning and the approximate word count.

We generate these baseline rewrites using two distinct models: our primary generation model, Qwen3.5-9B (Qwen Team, 2026) and GPT-5.5 (OpenAI, 2026). By evaluating the detector scores of these humanized outputs against the scores from our iterative attack, we can effectively quantify the added evasion capability provided by our targeted constraint search methodology.

2.5. Beam Search Design

For each of the 50 base prompts, we iteratively build a search tree of depth 5. The root node contains the initial unconstrained generation for the simple base prompt. From

that, we expand the tree with six nodes in layer 1, each prompting Qwen to generate a one-sentence constraint based on the previous prompt, generation output, detector score, and the respective constraint category for this node together with its definition. After appending the generated constraint to the prompt, we instruct Qwen with it and extract the corresponding new generation, on which we then run the detector. Therefore, every node except the root consists of one specific constraint based on the previous results, the corresponding new text generation, and its detection score. For the layers 2-5, we extend the three nodes with the lowest detector scores of the previous layer with six new nodes, respectively, which append the new constraint to the previous prompt and its constraints, resulting in a growing chain of up to five constraints in the last tree layer. Therefore, we perform a beam search of width 3, with one root node, six nodes in layer 1, and 18 nodes in deeper layers. This strategy aims to balance computational efficiency by being greedy with search exploration by allowing three parallel paths to be extended.

2.6. Coherence Judgment

To evaluate whether the chain of constraints leads to generations that lose their coherence, grammar, or meaning (e.g., by repeating the same word over and over again), we used Qwen3.5-9B once more for judging the coherence of the outputs of all nodes in all trees on a scale of 1-5. The prompt is shown in 10. We decided to judge coherence relative to the original prompt and its constraints. Specifically, if a constraint enforces the text to obtain an unnatural structure, style, or sound, the judge should not label it incoherent, as long as it is still as coherent as possible given the constraints.

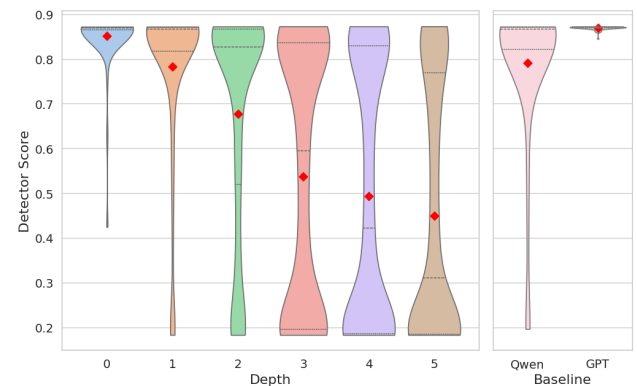


Figure 1. Distribution and mean of detector scores for the different tree depths. The depth corresponds to the number of constraints included in the prompt. On the right, we exhibit the two humanizing baseline score distributions.

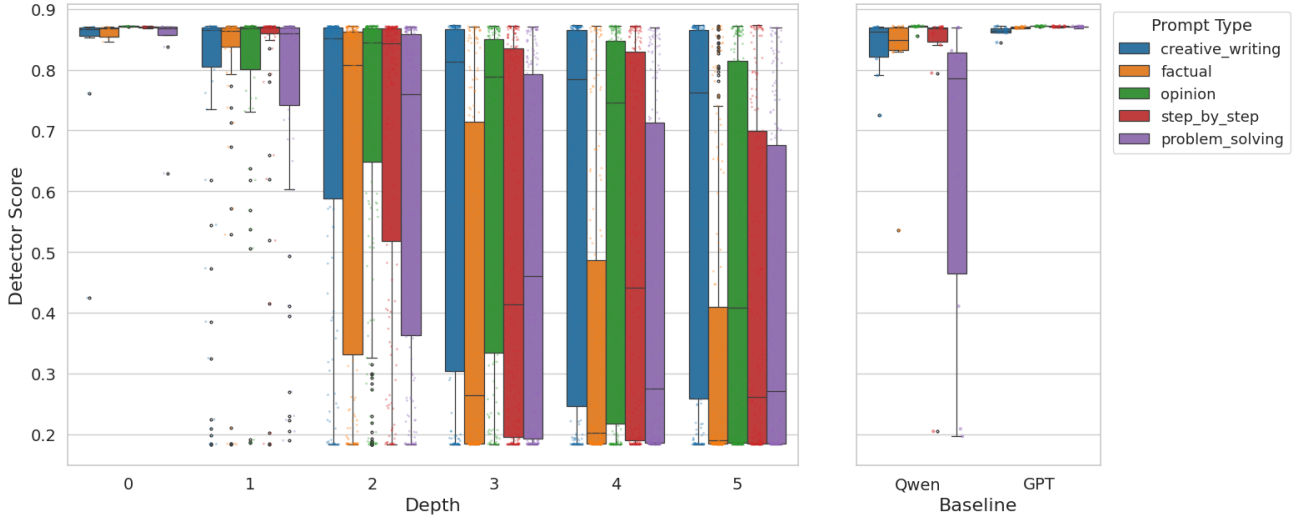


Figure 2. Distribution of detector scores for the different tree depths, sorted by prompt type. The depth corresponds to the number of constraints included in the prompt. On the right, we exhibit the two humanizing baseline score distributions.

3. Results

3.1. Effect of the Number of Constraints (Tree Depth)

Our analysis of 3,950 tree nodes (i.e., generated texts) in total demonstrates a clear inverse relationship between tree depth and the AI-detection probability as shown in Figure 1. In contrast, our humanizing baselines with both Qwen and GPT yielded high AI-detection probabilities, with the GPT model demonstrating high consistency (Mean: 0.869, σ : 0.005) and the Qwen model showing higher variance (Mean: 0.792, σ : 0.173). However, as our adversarial beam search iteratively applied constraints from depth 0 to depth 5, the procedure successfully navigated the search space to generate significantly more evasive text, reducing the mean AI-detection probability from 0.851 at depth 0 to 0.450 at depth 5. These findings indicate that, on average, extending the chain of constraints yields generations that are increasingly deviating from the typical distribution of machine-generated texts that our OOD detector is based on.

3.2. Sensitivity across the prompt domains

We found that the prompt type had an effect of the efficacy of the detection as shown in Figure 2. Creative writing outputs were easily detected as AI-generated—even with several prompt constraints, the detector still caught most of them. In contrast, constraining the output for factual and problem solving tasks led to a high count of detection failures. By depth 2 and 3, these tasks exhibit a large drop in detection scores. This could be due to the more rigid nature of factual tasks for which adding rules changes the writing pattern more drastically, making it harder for the detector to recognize the text as being AI-generated. In addition, this might also be due to the detector’s training

data which contained highly variable AI-generated creative texts, while factual texts were more standardized, leading to a distribution of lower variance that is easier to escape from by our attack.

3.3. Effect of Constraint Category

To understand the impact of specific constraint types, we analyzed the distribution and variance of detection scores immediately following the first constraint application (depth 1) as shown in Figure 3. Stylistic and mimicry constraints introduced the highest level of score volatility, as evidenced by the wide, elongated distribution of the violin plots. This indicates that these constraints are “high-risk, high-reward”: they have the potential to significantly lower detection scores, but also lead to unpredictable outputs that may occasionally result in higher detection scores. Conversely, categories such as structural and content-level constraints exhibit more compressed distributions, indicating a more predictable, albeit moderate, impact on the detector’s assessment.

Furthermore, we examined the impact of constraint combinations on the total detection score reduction between the root node and the leaves at depth 5. Figure 5 shows the relationship between the number of times a specific constraint category occurred in a full tree path from root to leaf and the corresponding total reduction of detector score. This plot enables one to interpret both the impact of multiple constraints of one category on the evasion, but also whether it is actually more beneficial to leave a constraint category out (occurrence 0). We observe that including one or multiple stylistic constraints reduces the final detector score on average, while mimicry constraints significantly improve

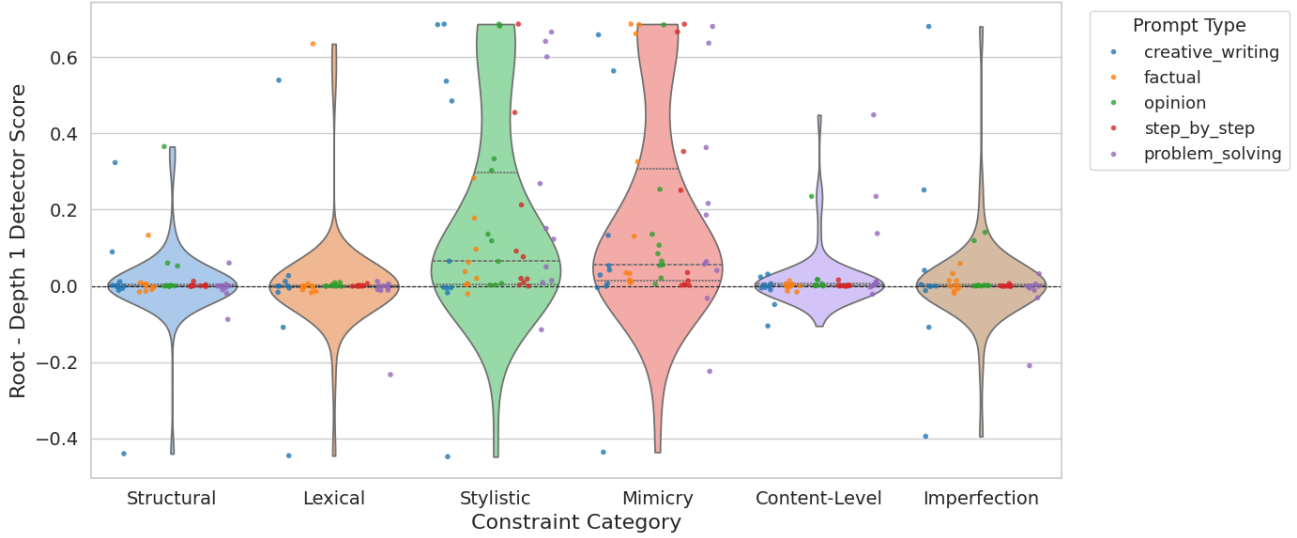


Figure 3. Relative improvement of the detector score for depth 1 (one constraint) nodes, sorted by constraint category and prompt type.

the detector evasion when they are included in the path. For content-level constraints, we observe that including multiple of them leads to higher score reductions on average, with the maximum being reached at an occurrence of 4. This analysis also reveals that multiple constraints from one category (e.g., 4 mimicry constraints) might not lead to a more human-like output but confuse the generation and perhaps also the detector model, because the constraints may interfere with each other or may be too hard to be followed.

3.4. Impact on Coherence

To understand if and to what degree the constraining affected the quality of the AI’s writing, we measured the coherence of the text. As shown in Figure 4, the automated coherence scores remained high at all levels of evasion. We observed clear horizontal bands at scores of 4 and 5, suggesting that the automated tool viewed most generations as logical and clear, regardless of the detector score. However, a manual review of a sample of these generated outputs revealed a different pattern. Although the automated tool labeled most generations as coherent, we observed instances where the text was actually nonsensical or logically broken. This indicates that while our method successfully lowered the detector scores, the automated coherence metric was not always able to identify when the underlying logic of the text had failed. Moreover, some constraints, despite being merely one sentence long, introduced multiple heavy rules which the generation LLM had to obey. Thus, adding multiple such constraints to the prompt inevitably leads to these types of nonsensical generations, which are still labeled as coherent because they follow the prompt and “try their best” in adhering to all those rules.

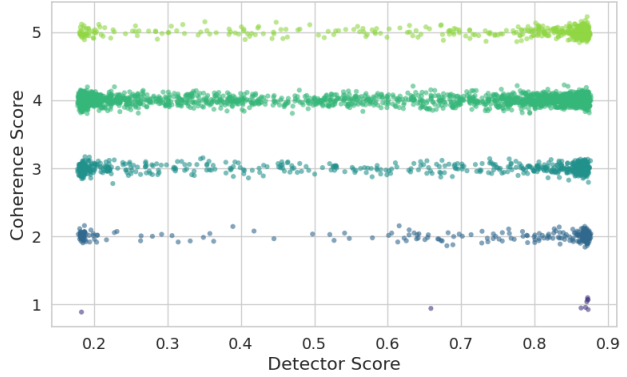


Figure 4. Relationship between detector scores and coherence scores. Coherence scores assigned by Qwen-as-a-judge.

4. Discussion

The results suggest that current detection classifiers rely heavily on identifiable stylistic priors that are inherent to unconstrained LLM output. When we impose multi-dimensional constraints (lexical, structural, etc.), the adversarial beam search restricts the model’s linguistic degrees of freedom. This forces the generation of syntactic patterns that differ from the classifier’s training distribution. This divergence is enough to lower the detector’s confidence, allowing the text to pass as human-authored as it departs from the AI’s default style.

We found that the model’s linguistic flexibility is domain-dependent. Factual and problem-solving tasks have strict logical requirements, which makes them easier to shape; adding constraints here forces the model to shift its output significantly. In contrast, creative writing prompts showed

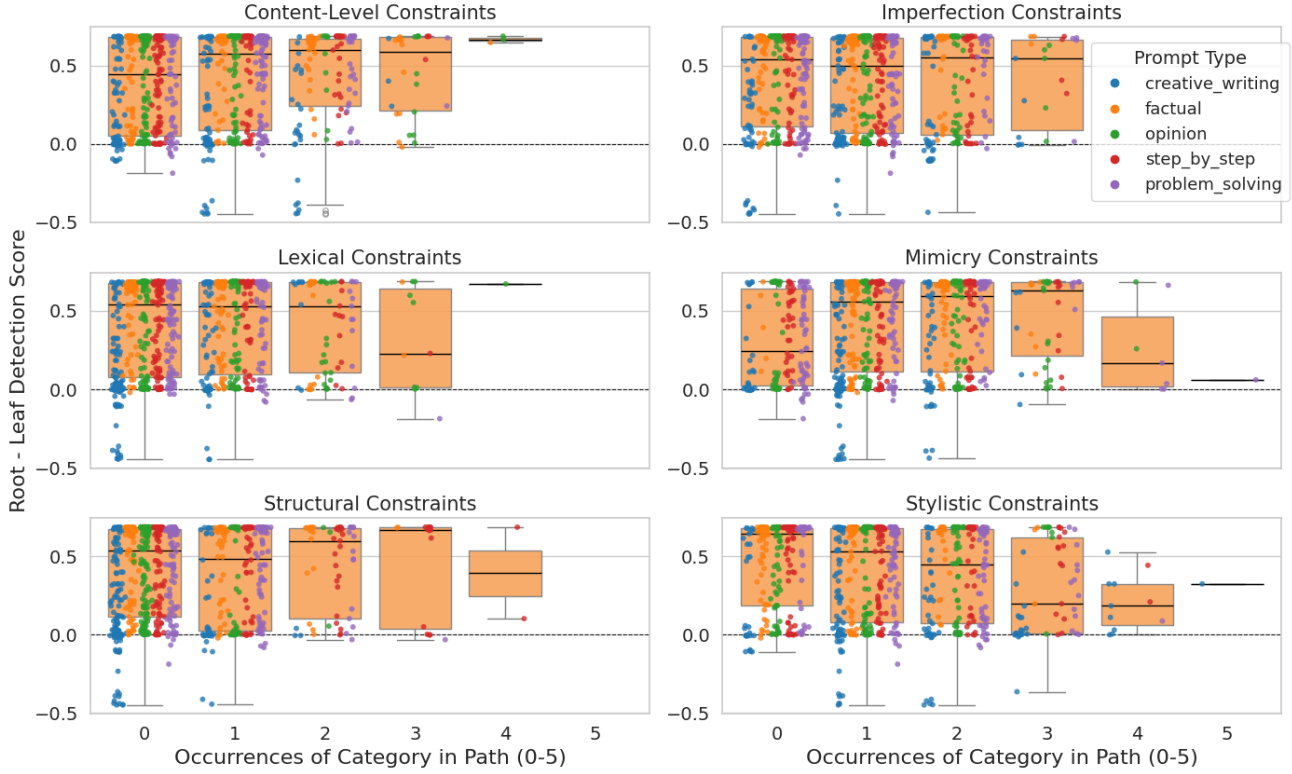


Figure 5. For each leaf node at the final tree layer (5 constraints), we trace back its path and count the occurrences of each constraint category. We plot the relative improvement of detector scores (root – leaf node) dependent on the number of occurrences of a category in the path. One leaf score can thus have contributions to multiple subplots.

high baseline resilience. This suggests that creative tasks have a stronger ingrained “semantic fingerprint”. Even when applying structural rules, the creative style is so robust that it resists these changes, rendering it much harder to hide the AI’s underlying patterns in creative domains compared to factual ones.

A key finding is that we can trick the detector while still maintaining high coherence scores according to automated tools. While the automated scores suggest the text remains logical, our human analysis showed that the adversarial process sometimes prioritizes evasion tactics that sacrifice narrative sense. This implies that while our method is successful at bypassing detectors, the system may inadvertently prioritize “tricking” the detector at the expense of human-perceived quality. Therefore, a low detection score does not always guarantee that the text is logically sound to a human reader.

5. Limitations & Future Work

There are several limitations to our work. For instance, we could only use a moderately powerful open-weight model, limiting the capability of finding successful constraints and generating truly diverse texts. In addition, although the au-

thors of our OOD detector (Zeng et al., 2025) emphasize its SoTA performance, using more powerful non-open-source detectors like GPTZero (Adam et al., 2026) trained on more domains and samples would perhaps change the results of our experiment, as it could have simply shifted the texts into a direction that is not well covered by the dataset used for training the DeepFake DSVDD we employed. It would also be interesting to distinguish multiple detectors that function very differently from each other and to expand the study to a different language which may enable more diversity.

Despite our coherence judge labeling most constrained generations as coherent, we observed plenty of incoherent generations when examining a sample of the leaf outputs; however, these are often due to the heavy constraints, which enforce the text to become unnatural and pointless. Therefore, a more careful and perhaps manual invention of constraints might reveal more significant insights, and it remains questionable how many of the generations with low detection score could really be consumed by humans without noticing their artificial origin or strange patterns. In conclusion, our study may not primarily reveal the LLM’s capabilities in generating human-like text, but the vulnerabilities of detectors to common sentence and word patterns that are broken by the constrained prompts, particularly in domains where

one would not expect these drastic shifts (e.g., factual).

6. Conclusion

Our work showed an example of applying multi-layered prompt constraints to effectively evade AI-content detectors. Our findings show that while standard AI output is easily flagged, when we use a constraint-stacking process we can significantly lower the probability of detection. This effectiveness is particularly strong in rigid factual domains, whereas creative tasks remain more resistant to these modifications. However, future work is needed to ensure that our results truly imply that LLMs contain hidden capabilities in generating human-like text that can be unlocked by intricate prompt design.

References

- Adam, G. A., Cui, A., Thomas, E., Napier, E., Shmatko, N., Schnell, J., Tian, J. J., Dronavalli, A., Tian, E., and Lee, D. Gptzero: Robust detection of llm-generated texts, 2026. URL <https://arxiv.org/abs/2602.13042>.
- Dugan, L., Hwang, A., Trhlík, F., Zhu, A., Ludan, J. M., Xu, H., Ippolito, D., and Callison-Burch, C. Raid: A shared benchmark for robust evaluation of machine-generated text detectors. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12463–12492, 2024.
- Li, Y., Li, Q., Cui, L., Bi, W., Wang, Z., Wang, L., Yang, L., Shi, S., and Zhang, Y. Mage: Machine-generated text detection in the wild, 2024.
- OpenAI. Gpt-5.5 system card, 2026. <https://openai.com/>.
- Paredes, J. L., Druck, G., Benson, B., and Smith, E. Ai now writes as many online articles as humans do. White paper, Graphite.io, May 2026. URL <https://graphite.io/five-percent/ai-now-writes-as-many-online-articles-as-humans-do>. Accessed: July 18, 2026.
- Qwen Team. Qwen3.5: Towards native multimodal agents, February 2026. URL <https://qwen.ai/blog?id=qwen3.5>.
- Reinhart, A., Brown, D. W., Markey, B., Laudénbach, M., Pantusen, K., Yurko, R., and Weinberg, G. Do llms write like humans? variation in grammatical and rhetorical styles, 2024. URL <https://arxiv.org/abs/2410.16107>.
- Zeng, C., Tang, S., Chen, Y., Shen, Z., Yu, W., Zhao, X., Chen, H., Cheng, W., and Xu, Z. Human texts are outliers: Detecting llm-generated texts via out-of-distribution detection, 2025. URL <https://arxiv.org/abs/2510.08602>.

Appendix

Structural Constraints: Structural constraints modify the overall shape, organization, and pacing of the text, including how ideas are ordered, how sentence lengths vary across the text, how sections differ in density or elaboration, and how the rhythm or flow changes from beginning to end, without prescribing specific content or wording.

Lexical Constraints: Lexical constraints modify the overall choice, distribution, and variation of words used in the text, including preferences for certain types of vocabulary, avoidance of particular lexical patterns, or shifts in word frequency, register, or specificity, without dictating the underlying content or structure.

Stylistic Constraints: Stylistic constraints shape the general tone, voice, and expressive character of the text, influencing how the writing feels or sounds—such as its formality, emotional coloration, narrative attitude, or rhetorical flavor—without prescribing specific words or altering the factual content.

Mimicry Constraints: Mimicry constraints direct the text to adopt the characteristic style, voice, or expressive tendencies of a particular author, persona, group, or dialect, guiding the overall stylistic imitation without requiring specific phrases or content from the source being mimicked.

Content-Level Constraints: Content-level constraints influence the distribution, emphasis, and balance of information across the text, shaping which aspects are expanded, minimized, prioritized, or reframed, without specifying the exact wording or stylistic execution.

Imperfection Constraints: Imperfection constraints introduce controlled irregularities that emulate natural human variability—such as slight inconsistencies, mild redundancies, subtle awkwardness, or uneven phrasing—without compromising overall clarity or altering the intended meaning of the text.

Figure 6. Constraint category definitions as given to the LLM.

Rewrite the following text to sound more human, natural, and emotionally resonant. Avoid repetitive patterns, robotic phrasing, or excessive structure. Maintain the core message and meaning, and keep the length roughly the same (around 350 words).

Original Text:
{TEXT}

Humanized Version:

Figure 7. Prompt used for the humanizing baseline.

{PROMPT_WITH_CONSTRAINTS}

Please produce a coherent text of approximately {TARGET_LENGTH} words. Natural variation is allowed but keep the overall length comparable to this target.

Figure 8. Prompt used for the generation of text based on constrained prompts.

The following prompt was given to an LLM:
{CURRENT_PROMPT}

The LLM produced the following output:
{MODEL_OUTPUT}

An AI-text detector assigned the following probability that this output is AI-generated:
{DETECTOR_SCORE}

Your task is to propose one new constraint that will be added to the original prompt in order to reduce the AI-generated probability assigned by the detector.

The constraint must come from the following category: {CONSTRAINT_CATEGORY}.
This category is defined as: {CATEGORY_DEFINITION}.

Please output one single sentence that contains only the invented constraint, with no explanation, no justification, and no additional text.

Figure 9. Prompt used for the generation of a constraint.

The following prompt was given to an LLM:
{FULL_PROMPT}

The LLM produced the following output:
{LLM_OUTPUT}

Task: Rate how coherent the output is on a scale from 1 to 5 (1 = incoherent, 5 = perfectly coherent).
Guidelines: Judge coherence relative to the original prompt and its constraints. Consider logical flow, clarity of ideas, sentence-to-sentence connections, and whether the output stays on task. Penalize needless repetition, circular phrasing, or repeated fragments that do not advance meaning.
Output requirement: Return exactly one integer (1,2,3,4,or 5) and nothing else.
Your rating:

Figure 10. Prompt used for the coherence judge.

Creative Writing:

Write a scene where two strangers discover they have the same recurring dream.
 Describe a city where people's shadows have minds of their own.
 Tell a story about a letter that arrives fifty years too late.
 Invent a myth explaining why the stars flicker at night.
 Write a moment where a character hears music no one else can hear.
 Describe a marketplace that sells emotions in glass bottles.
 Create a tale about a door that appears only when someone feels lost.
 Write a scene where time briefly runs backward for everyone except one person.
 Describe a world where memories can be traded like currency.
 Tell a story about a lighthouse that signals messages from another dimension.

Factual:

Explain how photosynthesis converts light energy into chemical energy.
 Define the term 'geostationary orbit' and describe its key properties.
 Summarize the causes and consequences of the 2008 financial crisis.
 Describe how vaccines stimulate an immune response in the human body.
 Explain the difference between mitosis and meiosis.
 Define entropy in the context of thermodynamics.
 Describe how the CRISPR-Cas9 gene-editing system works.
 Explain why the sky appears blue during the day.
 Summarize the major functions of the human liver.
 Describe the structure and purpose of the United Nations.

Opinion:

Describe your personal opinion on whether technology makes life better or worse.
 Share a reflection on what makes a book truly memorable to you.
 State your view on the importance of taking risks in life.
 Give your opinion on whether people are naturally optimistic or pessimistic.
 Reflect on what you believe defines a meaningful friendship.
 Share your personal take on why humans enjoy storytelling.
 State your view on whether routine helps or hinders creativity.
 Give your opinion on what makes a city feel alive.
 Reflect on how you feel about the balance between ambition and contentment.
 Share your view on whether nostalgia is helpful or harmful.

Step-by-Step:

Provide a step-by-step guide for organizing a cluttered workspace.
 Give a step-by-step explanation of how to back up files to an external drive.
 Write a step-by-step process for brewing the perfect cup of coffee.
 Create a step-by-step tutorial for setting up a new email account.
 Explain the step-by-step method for changing a flat tire.
 Outline a step-by-step workflow for planning a weekly schedule.
 Give a step-by-step recipe for making homemade pancakes.
 Provide a step-by-step guide for resetting a forgotten password.
 Write a step-by-step process for cleaning and maintaining a keyboard.
 Explain the step-by-step instructions for installing a software update.

Problem Solving:

Suggest the most efficient way to diagnose a laptop that suddenly won't boot.
 Explain how you would prioritize tasks when managing an overloaded project schedule.
 Describe the best approach to resolve a conflict between two team members.
 Outline how to identify the root cause of a sudden drop in website traffic.
 Propose a strategy to reduce customer churn for a subscription service.
 Explain how to choose the optimal route for a delivery driver facing road closures.
 Describe how to troubleshoot a home Wi-Fi network with intermittent outages.
 Suggest a plan to minimize downtime during a server migration.
 Explain how to decide whether to repair or replace a malfunctioning appliance.
 Propose a method to allocate a limited budget across competing priorities.

Figure 11. The 50 base prompts sorted by category.

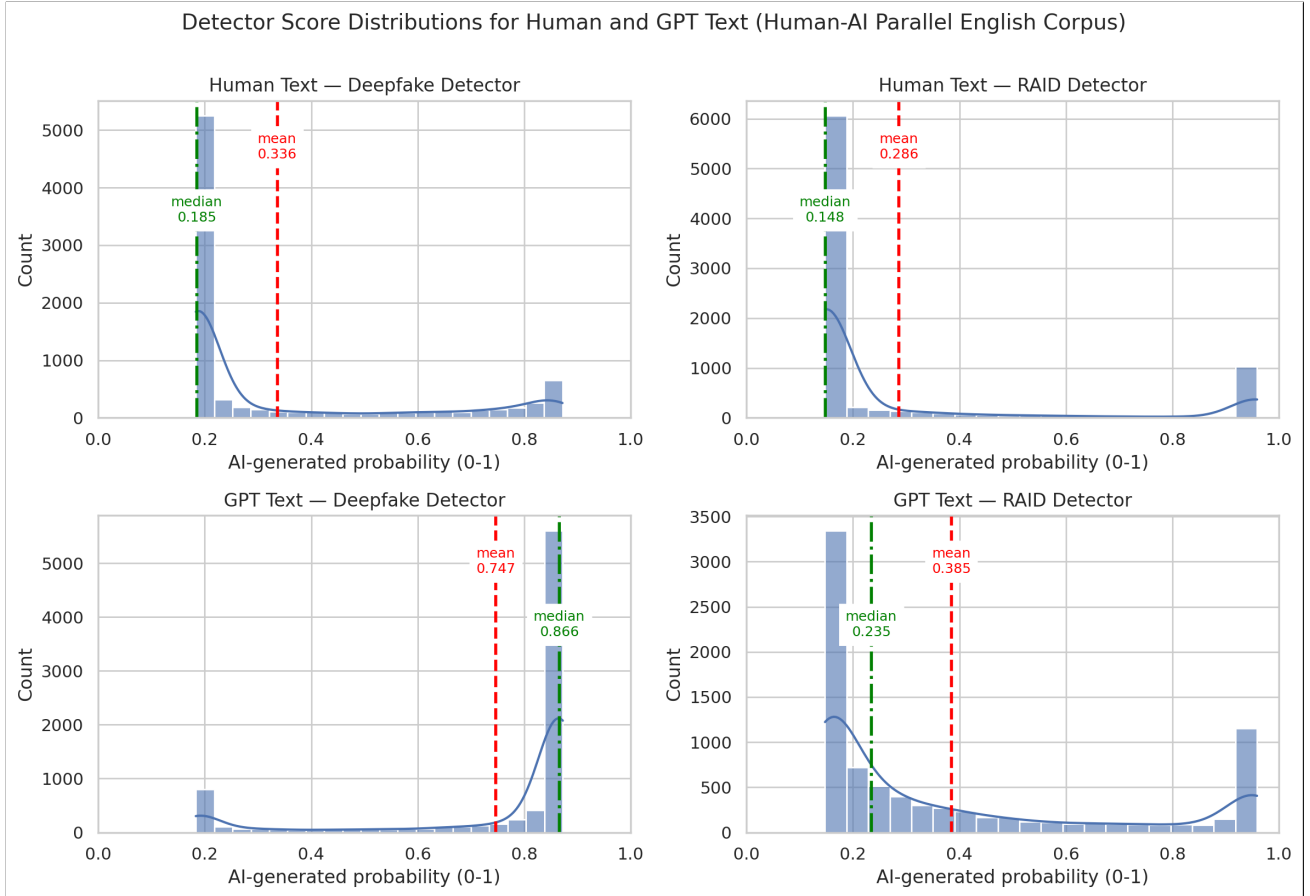


Figure 12. Prior to using our detector, we evaluated it against the Human-AI Parallel English Corpus (Reinhart et al., 2024).